

A property is an excellent example of abstraction. As you saw earlier, with properties the client programmer can use a syntax that makes their intent clear (to get and set a value in an object). The implementation in the methods that are called as a result can be much more complicated, but is totally hidden from the client code.

Summary

Encapsulation is the first major tenet of object-oriented programming, allowing classes to hide their implementation and data from client code and ensuring that a class provides all the functionality that a client needs in a single place.

A key concept of OOP is that objects own their data, and that's why I recommend that you provide getter and setter methods if you want client code to access the data held in an instance variable. Python does allow direct access to instance variables using dot syntax, but I strongly encourage you to stay away from this syntax for the reasons laid out in this chapter.

There are conventions for marking instance variables and methods as private, using a leading underscore or double underscore depending on the level of privatization you require. As a compromise, Python also allows the use of the `@property` decorator. This makes it appear as if the client code is able to access an instance variable directly, while behind the scenes Python turns such references into calls to the decorated getter and setter methods in the class.

The `pygwidgets` package provides many good examples of encapsulation. As a client programmer, you see a class from the outside and work with the interfaces that the class provides. As a class designer, abstraction—handling complexity by hiding details—helps you design a good interface by considering the interface of the class from the client's point of view. However, in Python, you often have the source code available so that you can look at the implementation if you wish.